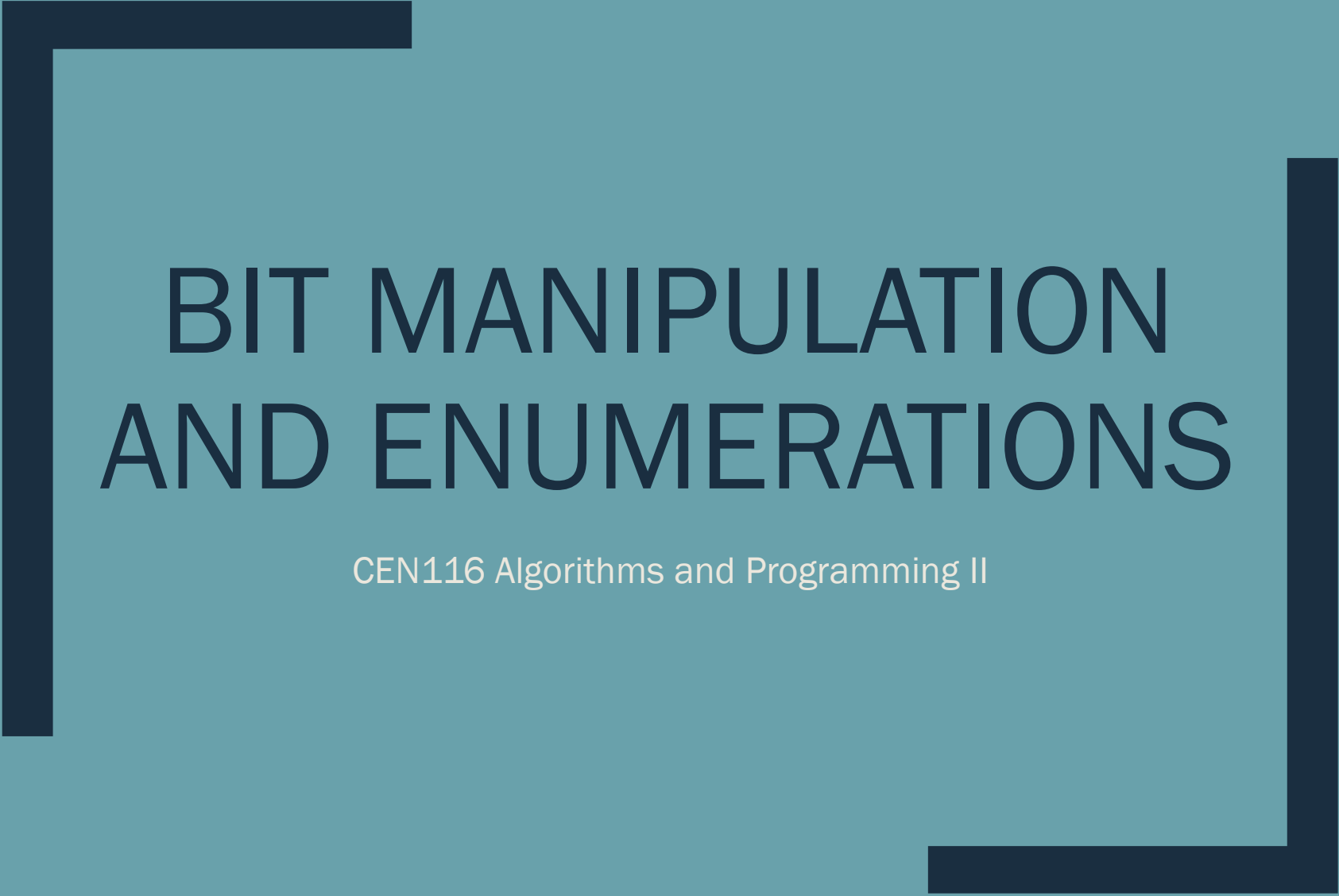




BIT MANIPULATION AND ENUMERATIONS

CEN116 Algorithms and Programming II

Bitwise Operators

- Computers represent all data internally as sequences of bits.
- Each bit can assume the value 0 or the value 1.
- On most systems, a sequence of 8 bits forms a byte—the typical storage unit for a variable of type `char`.
- Other data types are stored in larger numbers of bytes.
- The bitwise operators are used to manipulate the bits of integral operands both signed and unsigned.
- Unsigned integers are normally used with the bitwise operators.

Bitwise Operators

- The bitwise operator discussions in this section show the binary representations of the integer operands.
- Because of the machine-dependent nature of bitwise manipulations, these programs may not work correctly on your system.
- The bitwise operators are
 - bitwise AND (&)
 - bitwise inclusive OR (|)
 - bitwise exclusive OR (^)
 - left shift (<<)
 - right shift (>>)
 - complement (~)

Bitwise Operators

- The bitwise AND, bitwise inclusive OR and bitwise exclusive OR operators compare their two operands bit by bit.
- The bitwise AND operator sets each bit in the result to 1 if the corresponding bit in both operands is 1.
- The bitwise inclusive OR operator sets each bit in the result to 1 if the corresponding bit in either (or both) operand(s) is 1.
- The bitwise exclusive OR operator sets each bit in the result to 1 if the corresponding bit in exactly one operand is 1.

Bitwise Operators

- The left-shift operator shifts the bits of its left operand to the left by the number of bits specified in its right operand.
- The right-shift operator shifts the bits in its left operand to the right by the number of bits specified in its right operand.
- The bitwise complement operator sets all 0 bits in its operand to 1 in the result and sets all 1 bits to 0 in the result.

Bitwise Operators

Operator		Description
&	bitwise AND	Compares its two operands bit by bit. The bits in the result are set to 1 if the corresponding bits in the two operands are <i>both</i> 1.
 	bitwise inclusive OR	Compares its two operands bit by bit. The bits in the result are set to 1 if <i>at least one</i> of the corresponding bits in the two operands is 1.
^	bitwise exclusive OR (also known as bitwise XOR)	Compares its two operands bit by bit. The bits in the result are set to 1 if the corresponding bits in the two operands are different.
<<	left shift	Shifts the bits of the first operand left by the number of bits specified by the second operand; fill from the right with 0 bits.
>>	right shift	Shifts the bits of the first operand right by the number of bits specified by the second operand; the method of filling from the left is machine dependent when the left operand is negative.
~	complement	All 0 bits are set to 1 and all 1 bits are set to 0.

Displaying an Unsigned Integer in Bits

- When using the bitwise operators, it's useful to display values in binary to show the precise effects of these operators.
- The next program prints an unsigned int in its binary representation in groups of eight bits each for readability.
- For the examples in this section, we assume an implementation where unsigned ints are stored in 4 bytes (32 bits) of memory.

```
#include <stdio.h>

void displayBits(unsigned int value); // prototype

int main(void)
{
    unsigned int x; // variable to hold user input
    printf("%s", "Enter a nonnegative int: ");
    scanf("%u", &x);
    displayBits(x);
}

// display bits of an unsigned int value
void displayBits(unsigned int value)
{
    // define displayMask and left shift 31 bits
    unsigned int displayMask = 1 << 31;
    printf("%10u = ", value);
    for (unsigned int c = 1; c <= 32; ++c) {
        putchar(value & displayMask ? '1' : '0');
        value <<= 1; // shift value left by 1
        if (c % 8 == 0) { // output space after 8 bits
            putchar(' ');
        }
    }
    putchar('\n');
}
```


Example Output

- Enter a nonnegative int: 1
1 = 00000000 00000000 00000000 00000001
- Enter a nonnegative int: 65000
65000 = 00000000 00000000 11111101 11101000
- Enter a nonnegative int: 1234567890
1234567890 = 01001001 10010110 00000010 11010010

Displaying an Unsigned Integer in Bits

- Function `displayBits` uses the bitwise AND operator to combine variable value with variable `displayMask`
- Often, the bitwise AND operator is used with an operand called a mask—an integer value with specific bits set to 1.
- Masks are used to hide some bits in a value while selecting other bits.
- In function `displayBits`, mask variable `displayMask` is assigned the value
 - $1 \ll 31$ (100000000 00000000 00000000 00000000)

Displaying an Unsigned Integer in Bits

- The left-shift operator shifts the value 1 from the low-order (rightmost) bit to the high-order (leftmost) bit in displayMask and fills in 0 bits from the right.
- `putchar(value & displayMask ? '1' : '0');` determines whether a 1 or a 0 should be printed for the current leftmost bit of variable value.
- When value and displayMask are combined using `&`, all the bits except the high-order bit in variable value are “masked off” (hidden), because any bit “ANDed” with 0 yields 0.

Displaying an Unsigned Integer in Bits

- If the leftmost bit is 1, `value & displayMask` evaluates to a nonzero (true) value and 1 is printed—otherwise, 0 is printed.
- Variable `value` is then left shifted one bit by the expression `value <<= 1` (this is equivalent to `value = value << 1`).
- These steps are repeated for each bit in unsigned variable `value`.

Bitwise Assignment Operators

- Each binary bitwise operator has a corresponding assignment operator.

Bitwise assignment operators

<code>&=</code>	Bitwise AND assignment operator.
<code> =</code>	Bitwise inclusive OR assignment operator.
<code>^=</code>	Bitwise exclusive OR assignment operator.
<code><<=</code>	Left-shift assignment operator.
<code>>>=</code>	Right-shift assignment operator.

Enumeration Constants

- An enumeration, introduced by the keyword `enum`, is a set of integer enumeration constants represented by identifiers.
- Values in an `enum` start with 0, unless specified otherwise, and are incremented by 1.
- For example, the enumeration
 - *`enum months {JAN, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC};`*
creates a new type, `enum months`, in which the identifiers are set to the integers 0 to 11, respectively.

Enumeration Constants

- To number the months 1 to 12, use the following enumeration:
 - *enum months {JAN = 1, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC};*
- Because the first value in the preceding enumeration is explicitly set to 1, the remaining values are incremented from 1, resulting in the values 1 through 12.
- The identifiers in an enumeration must be unique.
- The value of each enumeration constant of an enumeration can be set explicitly in the definition by assigning a value to the identifier.

Using an Enumeration

```
#include <stdio.h>

// enumeration constants represent months of the year
enum months {JAN = 1, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC};

int main(void)
{
    // initialize array of pointers
    const char *monthName[] = { "", "January", "February", "March",
        "April", "May", "June", "July", "August", "September", "October",
        "November", "December" };
    // loop through months
    for (enum months month = JAN; month <= DEC; ++month) {
        printf("%2d%11s\n", month, monthName[month]);
    }
}
```


Using an Enumeration

1	January
2	February
3	March
4	April
5	May
6	June
7	July
8	August
9	September
10	October
11	November
12	December

References

- C How to Program, 8ed, by Paul Deitel and Harvey Deitel, Pearson, 2016.